Career Plan Reference

Systems depth -> Robotics skills that redeploy
Tesla Optimus as north star, not a lock-in

Prepared for: Chandra Vig

Current: Poly video devices -- C++ and C drivers (NXP, Lontium, Qualcomm, Ambarella CV7).

Dream: Tesla Optimus, but only via skills that still hire elsewhere.

Learning style: visual, interactive, analogies. Interview coding without AI during the attempt.

Inside this document

- Strategy in plain English
- Ranked saved LinkedIn jobs + roles to add
- 6-12 month roadmap and weekly rhythm
- Best visual / interactive learning resources with links
- Interview coding gym (live coding without AI)

1. Strategy

Analogy: your career is a toolbox, not a branded toolkit. Tesla Optimus is a dream workshop. Build tools that also work at Figure, Apptronik, Boston Dynamics-class teams, NVIDIA robotics, Amazon Robotics, OpenAI host systems, and strong embedded employers -- so if Tesla does not hire, nothing is wasted.

Double down (your moat)

- SoC bring-up and vendor driver craft (NXP / Qualcomm / Ambarella-class)
- Cross-layer debugging: firmware vs driver vs app vs silicon errata
- Real-time and camera/sensor pipelines (sync, latency budgets)
- Linux systems depth (Device Tree, IRQ/DMA models, concurrency)
- Clear design docs and war stories -- hard for AI to fake in interviews

Do not bet the farm on

- Deep CUDA / NVIDIA-only specialization as Plan A
- Tesla-only knowledge that does not map elsewhere
- Prompting AI instead of writing and explaining code live

Thin optional layers (10-20% time)

Edge perception on SoCs you already know, ROS 2 fluency, PID/control intuition, and light host I/O literacy (PCIe/DMA). GPU awareness is fine; CUDA expert track is optional later.

2. Ranked jobs (saved + add)

Ranked by: (A) fit with Poly/SoC/C++/drivers, (B) demand outside one company, (C) bridge toward humanoid/robotics without Tesla lock-in.

Tier 1 -- Prioritize first

1. OpenAI -- Host Systems Software Engineer
   Closest to what you already do (C/C++, Linux, drivers, bring-up, PCIe/DMA). Redeploys to robot compute stacks. Strong demand profile. BEST next target among your saved list.

2. NVIDIA -- Networking / Solution Engineer (host & networking systems)
   Customer + device stack; more openings and redeployability than pure CUDA driver roles.

3. Broadcom -- NOS / forwarding-plane software (C/C++, ASIC SDK)
   Deep systems + silicon SDK + customer communication. Stretch on networking; strong long-term demand.

Tier 2 -- Strong with one adjacency

4. NVIDIA -- Datacenter power / system software (real-time control flavor).
5. Cloudflare -- Senior Distributed Systems -- high liquidity; needs Go/Rust stretch.
6. Perplexity -- Storage / API platform -- excellent backup demand; less Optimus-direct.
7. evoke -- Principal Golang -- career insurance if you add Go; weaker robotics bridge.

Tier 3 -- Later / only if you retool

CUDA Driver / CUDA DL Systems (NVIDIA) -- prestige, narrower redeploy surface; after host/systems depth.
Google GDC Storage Staff -- strong infra career, weak robotics bridge from Poly today.
Agentic AI / AI Security platforms (JPMorgan, Bank of America, etc.) -- unnatural next step from device drivers unless you pivot hard.

Jobs to ADD (likely missing from your list)

Search these at Tesla AND peers (Figure, Apptronik, Agility, 1X, Amazon Robotics, NVIDIA Isaac, Apple Special Projects):

- Embedded Software Engineer -- Humanoid / Robotics
- Perception Engineer -- Edge / Embedded Vision (your Ambarella/Qualcomm story maps here)
- Systems / Host / Platform Engineer -- Robot Compute / BSP
- Firmware Engineer -- Sensors / Cameras / ISP
- Controls Software Engineer -- Real-time (if you like control loops)
- Autonomy Middleware Engineer -- ROS 2 / sensor plumbing

Search keywords: ROS 2, Device Tree, ISP, MIPI, Yocto, PREEMPT_RT, V4L2, GStreamer, motor control, humanoid.

Simple priority: deepen Poly craft -> target Host Systems + robotics platform/edge-perception roles -> keep Cloudflare/Perplexity as backup -> CUDA optional later.

3. 6-12 month roadmap

Months 1-3 -- Superpower + coding gym

Theme: SoC bring-up excellence + interview-ready C++.

Analogy: SoC as a factory floor -- CPU = foreman, DMA = forklifts, interrupts = alarm bells, drivers = shift supervisors between machines and workers.

- At Poly: hardest bring-up / stability / performance bugs; one short design note per month
- Lab: Bootlin free kernel/driver materials + Device Tree talk; optional BeagleBone Udemy course
- Visual warm-up: Branch Education SoC + CPU videos before dense reading
- Coding: 4-5 NeetCode problems/week in C++ -- you solve first, no AI on the attempt

Months 4-6 -- Robotics layer that redeploys

Theme: same hands, new factory -- robots instead of conference devices.

- Brian Douglas PID (animations) then Modern Robotics Coursera courses 1-2 with simulator
- ROS 2 free interactive path -- nodes as radio workers passing messages (prefer C++ modules)
- Edge vision: OpenCV Bootcamp + Embedded CV Coursera; Qualcomm Academy Robotics (free)
- Portfolio: one documented mini perception or bring-up project with latency-budget diagrams

Months 7-9 -- Host / I/O adjacency

Theme: how compute talks to the physical world at speed -- opens Host Systems + robot compute roles.

- PCIe/DMA mental models (visual explainers first); thin RoCE intro only if curious
- Optional thin GPU literacy (kernels, host-device copies) -- awareness, not CUDA career
- Start interviewing Tier 1; quantify Poly wins (boot time, crash rate, latency, bring-up days)

Months 10-12 -- Package + apply

- Resume story: Embedded systems -> robot / edge compute -- three war stories with diagrams
- Mix: ~50% robotics/embedded (incl. Optimus if open), ~30% host/systems, ~20% infra backup
- Mocks: live coding + systems design for device/robot platform

Weekly rhythm

3x coding (45-60 min) | 2x visual systems/robotics learning | 1x build | 1x short diagram/write-up

4. Learning resources (visual-first)

Prefer watch-and-do, simulators, and analogies. Dense books are backups, not day one.

A. Embedded Linux / drivers (P0)

Bootlin -- Kernel & Driver Development (free slides + labs)

Bootlin | ~40-80 h self-study

Lab-heavy; Device Tree, IRQ, DMA diagrams. NXP board option maps to Poly work.

https://bootlin.com/training/kernel/ | *https://bootlin.com/docs/*

Device Tree for Dummies -- Thomas Petazzoni

YouTube | ~45-60 min

Diagram-driven talk on how drivers bind to hardware -- bring-up literacy.

https://www.youtube.com/watch?v=m_NyYEBxfn8

LiveOverflow -- How Do Linux Kernel Drivers Work?

YouTube | ~30-45 min

Live demo of open/read/write hitting a module -- everything-is-a-file made visible.

https://www.youtube.com/watch?v=juGNPLdjLH4

Udemy -- Embedded Linux Step by Step (BeagleBone)

Udemy | ~~16 h + projects

Watch-and-do on real board: boot flow, I2C, hardware walkthroughs.

https://www.udemy.com/course/embedded-linux-step-by-step-using-beaglebone/

Linux Foundation LFD435 -- Developing Embedded Linux Device Drivers

Linux Foundation (paid) | ~4 days

Instructor-led labs; production driver APIs.

https://training.linuxfoundation.org/training/developing-embedded-linux-device-drivers/

B. Architecture / OS (visual intuition)

Branch Education -- How SoCs Work / How CPUs Work

YouTube | ~35-50 min each

Best-in-class 3D teardowns: ISP, NoC, DRAM, camera path -- Ambarella/Qualcomm intuition.

Search Branch Education How do System on a Chip Work / How do CPUs Work on YouTube

Ben Eater -- Build an 8-bit computer

YouTube + eater.net | ~10-15 h watch

LEDs and breadboards make datapaths physical.

https://eater.net/8bit

Nand2Tetris

nand2tetris.org | ~selective weeks

Interactive emulators: build CPU to OS. Project-based.

https://www.nand2tetris.org/

CS:APP labs (CMU)

csapp.cs.cmu.edu | ~40-80 h selective

Bomblab / buflab / cachelab / malloclab -- systems interview gym.

https://csapp.cs.cmu.edu/

C. Robotics (redeployable beyond Tesla)

Modern Robotics Specialization (Northwestern)

Coursera (~4.7 stars) | ~~4 months part-time or selective

Simulator + coding; kinematics/control with a robot you can see move.

https://www.coursera.org/specializations/modernrobotics

Brian Douglas -- PID / Control Systems

YouTube | ~2-4 h core

Gold-standard animations (cruise-control analogies) before heavy math.

https://www.youtube.com/user/ControlLectures

ROS 2 -- free interactive paths

Web / GitHub | ~40-50 h+

RViz + Gazebo: nodes as workers on a radio network. Prefer C++ modules.

Search: Albanna ROS2 interactive tutorial | openAMRobot ROS2 complete course

Packt -- Mastering ROS 2 (Coursera)

Coursera | ~weeks

Gazebo / Nav2 / MoveIt style workflows -- robotics is visual by nature.

https://www.coursera.org/specializations/packt-mastering-ros-2-for-robotics-programming

Underactuated Robotics (MIT / Tedrake)

underactuated.csail.mit.edu | ~selective chapters

Interactive figures + Drake. AFTER PID + Modern Robotics intro.

https://underactuated.csail.mit.edu/

D. Edge vision / your SoC story

Qualcomm Academy -- AI for Qualcomm Robotics (FREE)

Qualcomm Academy | ~~2 h + practice

Camera/video architecture, ROS demos -- matches Qualcomm adjacency.

https://www.academy.qualcomm.com/course-catalog/AI-for-Qualcomm-Robotics

Computer Vision with Embedded Machine Learning

Coursera (~4.8 stars) | ~~3 weeks @ 10h

Colab + deploy-to-board; edge/TinyML -- not CUDA-primary.

https://www.coursera.org/learn/computer-vision-with-embedded-machine-learning

OpenCV Bootcamp (free)

OpenCV University | ~~3 h

Short videos + Colab notebooks.

https://opencv.org/university/free-opencv-course/

Ambarella Developer Zone

ambarella.com/developer | ~ongoing

CVflow tutorials (SDK often gated). Closest public portal for CV7-class work.

https://www.ambarella.com/developer/

E. Optional thin networking

Marvell RDMA lightboard + short RoCE video

Marvell / YouTube | ~20-40 min

Lightboard diagrams only -- skip deep RDMA manuals unless targeting HPC networking.

Search: Marvell RDMA lightboard | RoCE short explainer on YouTube

5. Interview coding (no AI on the attempt)

Interviewers need YOU to think, write, and debug under pressure. Use AI later to review -- never during the timed practice that mimics the interview.

Analogy: DSA patterns are kitchen recipes -- NeetCode shows the technique; then you cook without looking. Systems questions are explaining why the oven smoked -- you need a model of heat, vents, and timers (memory, threads, I/O), not a memorized speech.

Primary gym

NeetCode 150 (C++)

neetcode.io + YouTube | ~8-12 weeks @ 1-2/day
Pattern videos with diagrams; practice in-browser, then re-solve offline without AI.
https://neetcode.io/

Educative -- Grokking Coding Interview Patterns in C++

Educative | ~20-40 h
Interactive pattern illustrations + in-browser code.
https://www.educative.io/courses/grokking-coding-interview-in-cpp

Educative -- Modern C++ Concurrency (Rainer Grimm)

Educative | ~~14 h
Threads, mutexes, atomics, memory model -- systems interview staple.
https://www.educative.io/courses/modern-cpp-concurrency-in-practice-get-the-most-out-of-any-machine

LeetCode (arena)

leetcode.com | ~ongoing
Follow NeetCode roadmap; tags: bit manipulation, design, concurrency.
https://leetcode.com/

Systems / embedded checklist (paper or whiteboard)

- Pointers, ownership, RAII, rule of 0/3/5
- Alignment, packing, endianness, volatile vs atomic
- MMIO, DMA completion, ISR -> thread handoff, ring buffers
- Mutex/CV vs lock-free; priority inversion; memory ordering basics
- Linux: processes/threads, virtual memory, epoll basics, driver lifecycle
- A crisp SoC bring-up story (cross-compile, Device Tree, a failure you rooted out)

Optional extras

- CS:APP labs -- https://csapp.cs.cmu.edu/
- Casey Muratori Performance-Aware Programming -- https://www.computerenhance.com/p/table-of-contents
- InterviewReady for host/cloud design rounds -- https://interviewready.io/

Bottom line

Deepen what you already do at Poly. Add robotics and edge-perception layers any humanoid company would recognize. Keep Host Systems roles as the strongest near-term target from your saved list. Keep CUDA optional. Practice coding like the interview is a blank editor -- because it will be.

If Tesla Optimus opens a door, you are ready with redeployable skills. If not, the same plan still lands excellent systems and robotics seats.

Resources verified Sep 2026. Links and ratings change; search the course title if a URL moves. Personal learning plan -- not an offer from any employer.